DevOps Project Report

Book Recommendation API

Automated CI/CD Pipeline with Kubernetes

Student: **Rana Romdhane** Date: January 14, 2026

Project Overview

Built a **Book Recommendation API** with fully automated deployment. When code is pushed to GitHub, it automatically tests, scans for security issues, builds a container, and deploys to Kubernetes - zero manual steps required!

1 Architecture

1.1 The Application

What it does: REST API that provides book recommendations based on user preferences (genre, rating, page count).

Tech Stack: • Node.js 18 + Express.js
• 10 sample books in memory
• Rate limiting (100 req/15min)
• Helmet.js security headers
• Health check endpoint

1.2 Containerization

Docker Setup: • Base: node:18-alpine (lightweight)
• Non-root user for security
• Health checks every 30s
• Layer caching for fast builds

Why containers? Works identically everywhere - my laptop, servers, cloud.

1.3 Kubernetes Deployment

Configuration: • 2 initial replicas (copies)
• CPU: 100-200m — RAM: 128-256Mi
• Auto-scaling: 2-5 pods at 70% CPU
• NodePort service (port 30001)

Benefits: Auto-restart on failure, automatic scaling, load balancing.

2 CI/CD Pipeline

Trigger: Every push to GitHub

2.1 Stage 1: Testing

• Install dependencies (npm ci)
• Run Jest tests + coverage
• Security audit (npm audit)
• Upload coverage reports

2.2 Stage 2: Security Scan

Trivy Scanner:
• Checks code dependencies for vulnerabilities
• Scans configuration files
• Uploads results to GitHub Security tab

2.3 Stage 3: Build & Deploy

Process:
1. Build Docker image with caching
2. Scan container with Trivy
3. Push to Docker Hub (SHA + main tags)
4. Ready for Kubernetes deployment

Optimization: Build time reduced from 3min to 45sec using cache!

3 Observability

3.1 Prometheus (Metrics)

Collects data every 10 seconds: • Request count & duration
• Error rates
• CPU/memory usage
• Custom business metrics

3.2 Jaeger (Tracing)

Tracks request journey: • Each request gets unique trace ID
• Shows where time is spent
• Identifies slow operations
• Connects logs to traces

3.3 Grafana (Dashboards)

Visual monitoring: • Real-time request graphs
• Error percentages
• Response time (p50/p95/p99)
• System health overview

4 Security

Application: Rate limiting, input validation, secure headers, dependency scanning

Container: Minimal image, non-root user, vulnerability scanning

Kubernetes: Resource limits, health probes, auto-healing

5 Tools Used

Development & CI/CD

Backend: Node.js, Express.js
Testing: Jest, Supertest
CI/CD: GitHub Actions
Security: Trivy, npm audit
Container: Docker, Docker Hub

Infrastructure & Monitoring

Orchestration: Kubernetes (Minikube)
Metrics: Prometheus
Tracing: Jaeger + OpenTelemetry
Dashboards: Grafana

6 Key Learnings

1. Automation is Powerful
Initial setup took time, but now every deployment is automatic and consistent. No human errors!

2. Monitoring is Essential
Without Prometheus/Grafana, I wouldn't know if the app was slow or broken. Trace IDs made debugging 10x easier.

3. Security First
Trivy found 40+ vulnerabilities I didn't know about. Fixing early prevented future problems.

4. Containers Solve "Works on My Machine"
Docker ensures identical behavior everywhere - development, testing, production.

5. Kubernetes Auto-Scaling Works
During load testing, K8s automatically added pods when CPU hit 70%, then removed them when traffic dropped.

7 Project Workflow

Complete Flow

1. **Code Change** → Push to GitHub
2. **Auto-Test** → Jest runs all tests
3. **Security Scan** → Trivy checks vulnerabilities
4. **Build** → Docker creates container image
5. **Deploy** → Push to Docker Hub
6. **Run** → Kubernetes pulls and runs containers
7. **Monitor** → Prometheus/Grafana track health

Why This Matters

This is how real companies (Netflix, Spotify, Airbnb) deploy applications: • **Fast:** Deploy multiple times per day
• **Reliable:** Automated tests catch bugs
• **Scalable:** Handle traffic spikes automatically
• **Observable:** Know exactly what's happening
• **Secure:** Continuous vulnerability scanning

Conclusion

This project demonstrates a **complete modern DevOps pipeline** from code to production. Every aspect is automated: testing ensures quality, security scanning prevents vulnerabilities, Docker provides consistency, Kubernetes enables scaling, and comprehensive monitoring keeps everything visible.

Skills Demonstrated: Backend development (Node.js), automated testing (Jest), CI/CD (GitHub Actions), containerization (Docker), orchestration (Kubernetes), monitoring (Prometheus/Jaeger/Grafana), security scanning (Trivy), and infrastructure as code (YAML).

The result is a production-ready system that deploys reliably, scales automatically, and provides complete observability - exactly how modern tech companies operate.

Repository: github.com/RanaRomdhane/book-recommendation-api **Contact:** rana.romdhane@enicar.ucar.tn